Cloud Migration AI Agent Pipeline Milestone 2 Report

CSE598 Capstone Project Group 2 | November 8, 2026

P1 turns discovery records and confirmed constraints into a validated AWS migration wave plan. All 12 preliminary P1 runs passed validation under the documented exception policy. The stakeholder endorsed the planning direction and requested a more refined interface.

Basic Information

Student name	Email
Chaitanya Muzumdar	cmuzumda@asu.edu
Dhruvik Chetankumar Asmani	dasmani@asu.edu
Tejas Rajaram Uttare	tuttare@asu.edu
Vraj Ashokbhai Chaudhari	vchaud22@asu.edu

Stakeholder Ananth Sundararajan Sekar

Stakeholder meeting November 6, 2026 at 9:00 a.m. The stakeholder and all four Group 2 members attended.

Section 1 Stakeholder Follow Up and Final Deliverables

We walked through CSV normalization, the Neo4j graph, 3,000 GB wave planning, FinOps, the greedy baseline, and the planned chatbot. We reviewed preliminary results from the prototype. The stakeholder liked the chat approach and requested a web UI. He endorsed the planning direction and asked us to refine the UI and interaction flow. The team subsequently built the Streamlit interface described in Section 2.

Feedback	Team response	Next action
Build a web UI and refine the interaction.	Added constraint confirmation, exception review, a plan view, and graph Q&A around the chatbot.	Clarify the pilot, early requests, and review choices. Validate the revised flow with the stakeholder.
Apply 3,000 GB per wave.	Kept a configurable per-wave limit and route indivisible exceptions to review.	Confirm caller ordering. Model throughput and cutover windows separately. Model throughput and cutover windows separately.
Use synthetic data. Real discovery and costs are proprietary.	Kept the stakeholder sample and synthetic portfolios. Documented the cost assumptions.	Broaden the synthetic evaluation and retain clear cost labels.

Agreed final deliverables. A reproducible P1 web planner, a B0/B1/P1 evaluation, and a final report and presentation with a screenshot walkthrough. The scope is AWS rehost planning with static cost estimates.

Acceptance criteria. Zero dependency violations, undersized instances, or unapproved over-limit waves. Honor all feasible early requests and reconcile costs to the frozen catalog. Targets are 85% constraint accuracy, 90% completion over at least 10 runs, 60% utilization, and 4/5 for plan usefulness and explanations in blind review.

Section 2 Implemented Agentic System

The implemented workflow takes a discovery CSV and a confirmed configuration and returns a proposed AWS rehost plan with one wave assignment per included server. It also exports risk and sizing rationale, transfer exceptions, and a trace of validation and review decisions. The architecture keeps resource calculations in deterministic code and gives agents bounded tasks where language interpretation or recovery is useful.

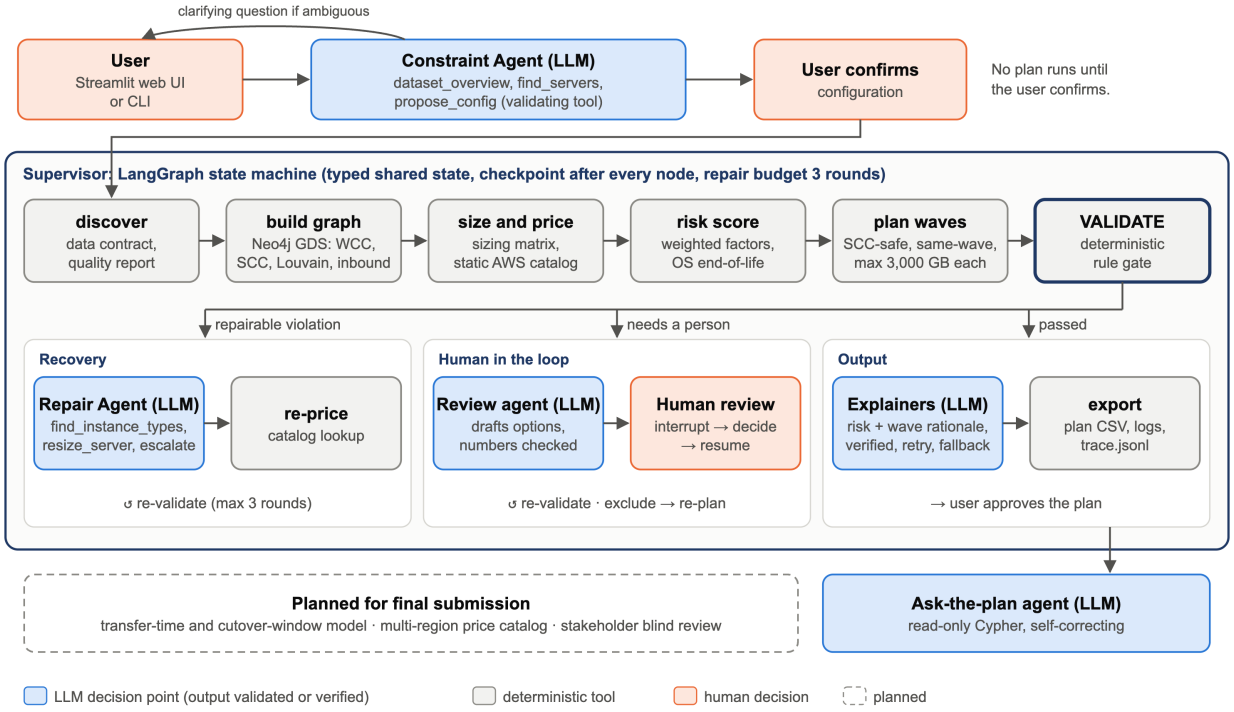


Figure 1. Implemented P1 workflow. Solid boxes show current components. The dashed box lists follow-up work. The 3,000 GB default allows reviewable exceptions for indivisible groups.

Discovery and dependency graph

The discovery step normalizes the CSV, records duplicate or missing data and unresolved connections, and builds one directed edge per distinct source–target relationship from the outbound records. Reverse adjacency and inbound degree come from those same edges. Neo4j Graph Data Science (GDS) calculates weakly connected components (WCC), strongly connected components (SCC), and Louvain communities. WCCs identify disconnected portfolios. SCCs keep cycles together. Communities guide which eligible groups to pack together. NetworkX supplies a local graph when Neo4j or GDS is unavailable.

Sizing risk and wave construction

Sizing starts from the stakeholder's allocation-based vCPU and RAM matrix. A frozen 17-SKU Linux On-Demand catalog for us-east-1 converts hourly compute prices to monthly estimates using 730 hours. The risk tool scores factors such as end-of-support operating systems, compliance tags, and inbound impact. The scheduler packs eligible SCC groups under the confirmed limit, preferring early workloads and their prerequisites, edges to groups already in the wave, unfinished communities, lower risk, and higher estimated savings. An optional pilot reserves wave 0 for eligible small, low-risk, isolated workloads.

Evidence Architecture and change history | Planner | Price catalog

Agent decisions and run state

Constraint capture. The Constraint Agent keeps the conversation in session memory. It calls dataset_overview and find_servers to ground names in the inventory, then sends a proposal through propose_config, which checks IDs, application names, cloud, and limit before storing typed settings. The user confirms the displayed settings. Those settings control the run.

Validation and repair. The LangGraph supervisor checkpoints typed per-run state in memory, including the confirmed config, server facts, edges, validation findings, repair history, and review decisions. The validator checks inventory coverage, dependency order, transfer, sizes, early requests, and applicable cost arithmetic. The Repair Agent reads violations and server allocations, looks up fitting catalog instances, then records an allowed resize or escalates. The supervisor reprices and validates again, with at most three repair rounds.

Human decisions. When the validator identifies a capacity exception or discovery gap, the review agent drafts guidance from supplied facts. Code sets the allowed actions. The interactive run pauses and resumes on the reviewer's choices. Approval records an exception, exclusion triggers a new schedule, and rejection stops the plan. Batch tests substitute a fixed policy. In-memory checkpoints support resume within the running session.

Explanations and questions. After validation, risk and wave explainers receive computed facts. Verifiers check cited factors, names, and numbers. A failed explanation gets a retry, followed by a template if needed. Export writes the CSV, graph, cost and validation summaries, review and repair logs, and trace.jsonl. Ask the Plan queries the exported graph using guarded read-only Cypher and shows returned rows. The agent can revise a query after a tool error.

P1 uses LangGraph for routing and interrupts, LangChain for tool calling, and qwen3-coder-30b-a3b-instruct through ASU Voyager at temperature 0. pandas and Pydantic handle data and typed settings. Streamlit and Altair provide the interface. Neo4j 5 with GDS and APOC runs in Docker, with NetworkX available for graph construction and scheduling offline.

Changes since Milestone 1

Milestone 1 design	Milestone 2 implementation or scope change
Chat-style constraint capture	Implemented a multi-turn agent plus a lightweight UI with visible confirmation and post-run review. The interface extends the chatbot after stakeholder feedback.
Community grouping and strict volume limit	Communities guide packing priority. SCCs remain the indivisible units and can move in the same wave as their predecessors. Oversized SCCs receive dedicated, blocking exceptions for review.
Supervisor and deterministic validation	Replaced the early prototype's subprocess chain and post-run validation with LangGraph routing, an in-run gate, guarded repair, and human review.
Static cost and limited prototype scope	Kept AWS rehost planning and static estimates. Transfer-time and cutover modeling, regional pricing, and formal stakeholder scoring remain follow-up work.

Evidence Supervisor and routing | Agent implementations | Run exports

Section 3 Preliminary Evaluation Results and Analysis

We compared three systems on the same complete inventories. Each evaluated example is one portfolio-level planning task. All 263 server rows across the four inventories were included in the batch comparison. Each baseline ran once per inventory. P1 ran three times per inventory.

Table 1. Planning datasets and coverage.

Dataset	Servers / edges	Source and planning challenge
D1	50 / 38	Stakeholder sample with synthesized transfer volumes, current costs, and early flags. Includes a 45 TB cycle and incomplete records.
D2	83 / 109	Seeded inventory with long chains and a 30-server data platform exceeding the cap as a portfolio.
D3 30	30 / 52	Small generated portfolio with shared services, fan-in, cycles, and missing fields.
D3 100	100 / 164	Larger generated portfolio with additional oversized workloads and more interdependent applications.

For D3, gpt-oss-120b generated the portfolio text. Seeded code added topology, defects, reference annotations, and early flags. We evaluated full portfolios without a train/test split. D3 500 (500 servers and 899 edges) and the blind review of three plans per system remain pending.

Baselines and execution conditions

B0 original stakeholder pipeline. Runs the stakeholder's six-stage Qwen and Neo4j workflow, which selects a cloud, sizes workloads, scores risk, and uses WCCs as wave groups. The wrapper supplies the shared input under B0's expected filename.

B1 greedy rule-only planner. Uses the same cleaning and cycle-safe packer as P1 with size-first priority, the original sizing matrix, and binary risk. It tests a simple packing heuristic without P1's priorities, repair, or agent calls.

P1 used AWS, CSV early flags, and no pilot. Its batch policy approved reported indivisible groups and acknowledged data gaps and infeasible early requests. The first two D1 P1 runs used 120-second attempts without a token cap. Later runs used 60 seconds and 6,000 tokens. B0 timed its shell and database workflow, B1 timed greedy planning, and P1 timed the full agent workflow.

Observed P1 timing ranges in seconds were D1 152.5–492.6, D2 204.1–321.4, D3 30 76.9–82.2, and D3 100 215.0–359.5. Each range summarizes three runs. Baseline variability is unknown because each baseline ran once per dataset. The D1 range combines the two model budgets.

Metrics and changes to the evaluation

We rescored the 12 saved plans with one validator. The strict result requires passing all validator checks with no exceptions to the 3,000 GB cap. A second pass allows the same exception for any system when a wave contains exactly one strongly connected component that alone exceeds the cap, including a single oversized server. We derived eligible groups from the input graph. Scheduling checks cover completeness, caller order, volume, and feasible early requests. AWS catalog fit and risk are separate checks. Utilization excludes exceptions, any pilot, and the final regular wave.

Evidence Evaluation harness | Datasets | B0 repository

Measured planning and agent results

Table 2. Common strict-cap scores and observed batch metrics. All plans had zero dependency-order violations.

Data	Sys.	Strict pass	Over / unreported	Under / non-AWS	Early %	Split %	Util. %	Waves	Time s
D1	B0	Fail	4 / 4	5 / 0	0.0	0.0	18.3	19	129.4
D1	B1	Fail	4 / 0	5 / 0	25.0	44.7	99.1	10	0.02
D1	P1	Fail	4 / 0	0 / 0	100.0	44.7	99.1	10	278.0
D2	B0	Fail	1 / 1	0 / 83	0.0	0.0	22.9	24	173.3
D2	B1	Fail	0 / 0	0 / 0	12.5	56.9	99.7	13	0.02
D2	P1	Pass	0 / 0	0 / 0	100.0	50.5	98.4	13	300.8
D3 30	B0	Fail	1 / 1	0 / 30	0.0	0.0	N/A	2	151.0
D3 30	B1	Fail	0 / 0	7 / 0	0.0	59.6	84.4	8	0.05
D3 30	P1	Pass	0 / 0	0 / 0	100.0	57.7	73.9	9	78.1
D3 100	B0	Fail	2 / 2	0 / 100	0.0	0.0	12.4	11	172.3
D3 100	B1	Fail	6 / 0	21 / 0	16.7	72.6	97.1	21	0.02
D3 100	P1	Fail	6 / 0	0 / 0	100.0	68.3	83.6	23	324.0

Over / unreported refers to the original outputs. Utilization is unavailable with fewer than two regular waves. Time summarizes three P1 runs per dataset (median) and one per baseline. Workflows and model budgets differ, so timings are descriptive.

Under the strict cap, P1 passed D2 and D3 30. Its only blockers on D1 and D3 100 were oversized groups. Under the common exception rule, P1 and B1 each qualified for 4 exceptions on D1 and 6 on D3 100. P1 then passed 4/4 and both baselines 0/4. B1 still missed early requests on all four datasets and had undersizing on three.

B0 failed scheduling checks on all four datasets even with sizing and risk excluded. Its Azure selections on three datasets are outside the AWS scope. Azure sizing fit was unevaluated. P1 met all feasible early requests and split fewer edges than B1 on three datasets, with lower utilization on those three. Both stayed above 60% utilization.

All 12 P1 runs completed under the batch review policy with matching wave assignments within each dataset. They logged 145 model calls, 244 tool calls, zero tool errors, and 18 verifier rejections. Rule-only P1 matched batch wave counts and split rates on D1 and D2. Disabling repair left five D1 servers undersized. Strict-earlier packing raised D2 from 13 to 33 waves.

Constraint parsing passed 30/31 requests (96.8%, target 85%), clarifying 6/7 ambiguous requests. On 27 D1 questions, P1 returned 25/27 correct row sets and 21/27 correct answers, versus 21/27 and 16/27 for B0-style GraphCypherQAChain. Query errors were 0 versus 2. Mean latency was 2.9 versus 4.1 seconds. Scoring used numeric tolerance, exact strings, or ID-set equality.

Evidence Raw planning runs | Constraint scores | Q&A scores

Section 4 Demo or Case Study

In Streamlit run sample-20261108-174139 on D1, the user requested a 3 TB limit, early migration for CRM and Payment Gateway, and a small low-risk pilot. The expected plan prioritizes feasible early requests and makes any capacity exception explicit.

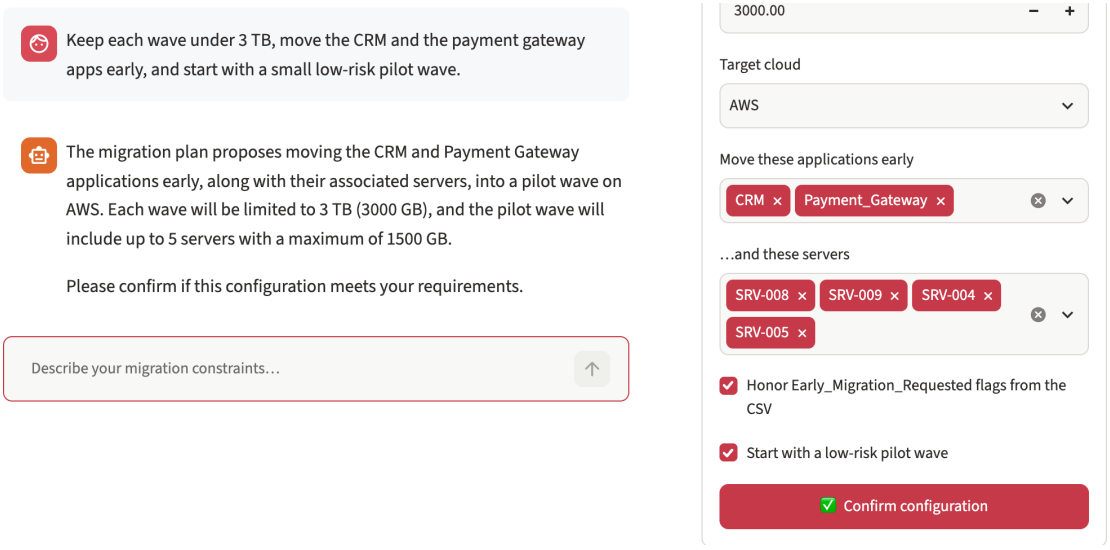


Figure 2. The Constraint Agent proposed a 3,000 GB AWS run with CRM and Payment_Gateway prioritized, four resolved server IDs, and pilot mode enabled.

The agent revised two rejected configuration proposals before the user confirmed the accepted one. Initial validation found five undersized instances and four oversized groups. The Repair Agent called list_violations, get_server, and find_instance_types. It resized SRV-018 (16 vCPU and 128 GB RAM) from m5.4xlarge to r5.4xlarge at $735.84 per month. The supervisor repriced and revalidated all five repairs.

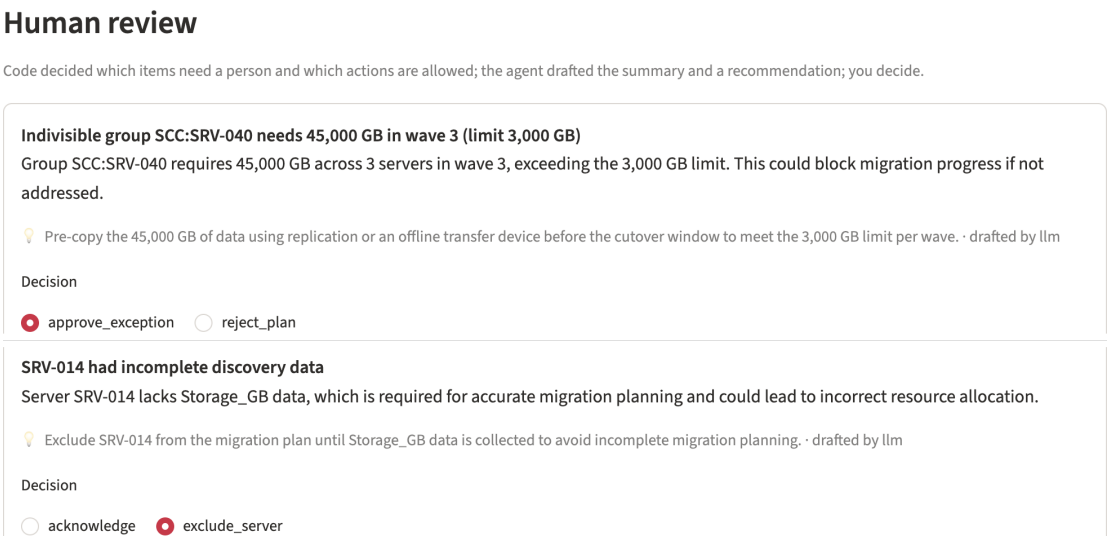


Figure 3. The review form surfaced the 45,000 GB warehouse group and the missing Storage_GB field for SRV-014. It presented review options and the agent's suggested actions.

The reviewer approved four exceptions and excluded SRV-014, SRV-031, and SRV-032 for missing data. LangGraph resumed, recomputed the schedule, and revalidated it. The warehouse exception still needs a cutover-time estimate.

Validated plan and graph question

The UI showed 11 waves, 25 of 36 remaining dependencies kept together, 90.3% utilization, and all feasible early requests on time. Four waves had approved exceptions. The verifier rejected a Wave 8 explanation for an unsupported number, and the retry passed. The user approved the plan in the UI and the run recorded the export location.

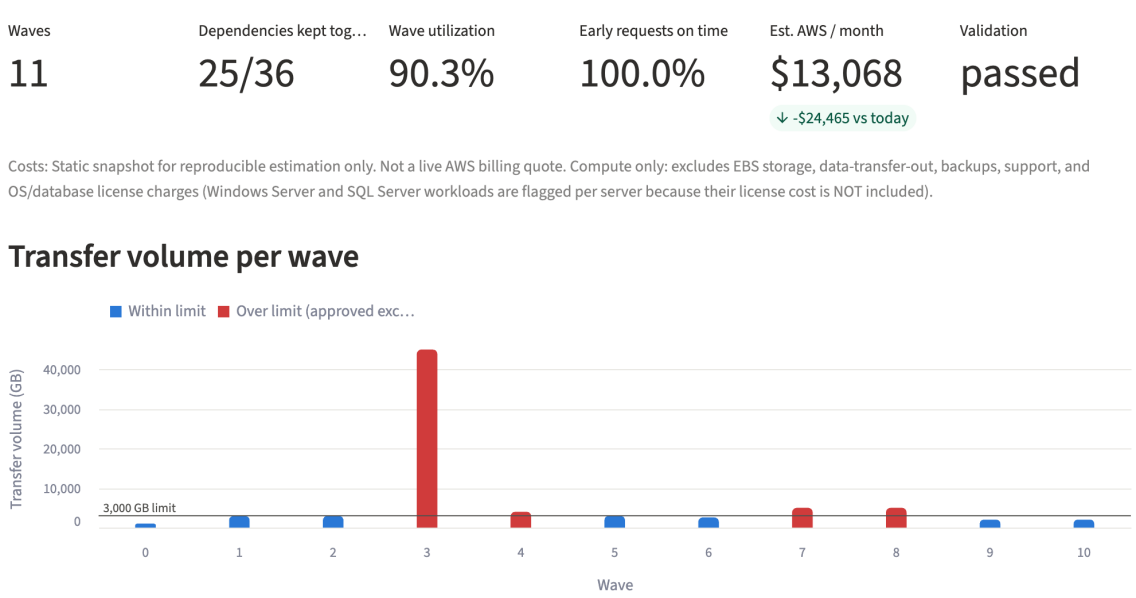


Figure 4. The final plan view shows validation passed and marks capacity exceptions in red. The AWS number is a static compute estimate. Section 5 explains a discrepancy in the displayed total after exclusion.

The user then asked which servers depend on hr-db-01 and which waves they are in. The Q&A agent called run_cypher with a read-only MATCH over incoming COMMUNICATES_WITH edges. Neo4j returned two rows, SRV-001 (hr-web-01) and SRV-002 (hr-app-01), both in wave 0. The agent answered from those rows in one query in 2.9 seconds.

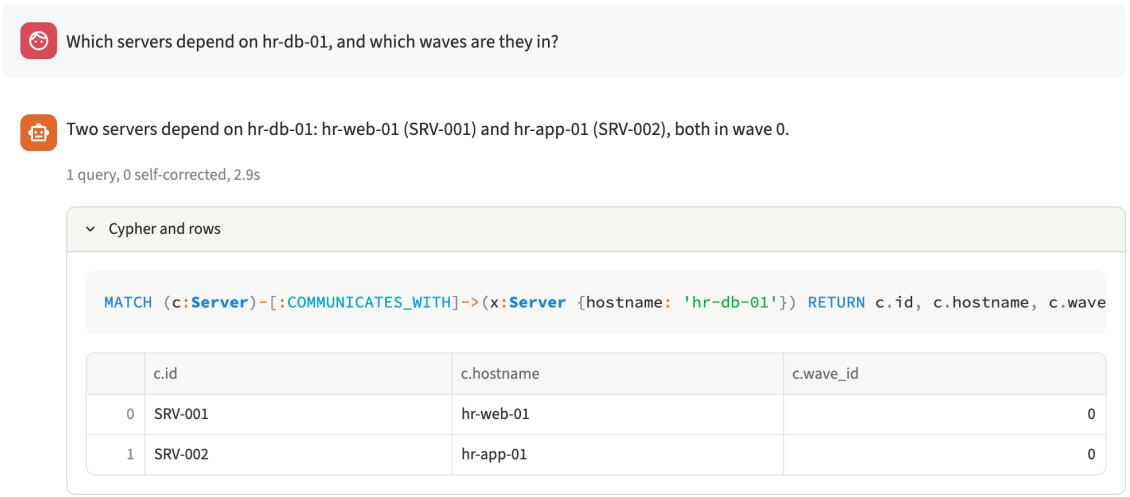


Figure 5. Ask the Plan displays the generated query and the two supporting rows. The result makes the derived inbound view directly inspectable.

Evidence Confirmed settings | Repair event log | Plan approval

Section 5 Failure Analysis and Current Limitations

Cost summary after excluding a server

An offline replay excluded the same three servers as the D1 demo and reproduced its 11 waves, 90.3% utilization, and 25 of 36 dependencies kept together. The replay validated with 47 scheduled servers and four approved exceptions. The active plan rows totaled $12,763.95 per month for AWS compute. The summary still showed $13,067.63, including $303.68 for the three excluded servers.

Exclusion routes to wave planning and validation, while the UI and exporter read the earlier cost totals. Validation checks active row prices. Recompute the totals after exclusion and test that current costs, AWS costs, and savings agree across the dashboard, summary, and active plan CSV.

Evidence Review routing | FinOps export | Plan view

Ambiguous constraint and misleading confirmation text

The constraint benchmark's only miss asked to move "ERP" early. The dataset matched ERP_System and ERP_Dev. The agent proposed both, although the expected action was to ask which application the user meant. The prompt limits clarification to multiple unrelated applications. The model may have interpreted the two environments as related. Add an explicit multiple-match check in the proposal path and test that all seven ambiguous prompts prompt clarification, with no new questions on the 24 clear cases.

The demo exposed a related wording problem. The chat reply implied that CRM and Payment Gateway would move into the pilot, while the structured form stored early requests and pilot selection separately. The planner excludes early-related groups from pilot selection. Build the confirmation summary from the validated fields and explain that the pilot is selected during planning. Ask a reviewer to identify the early set and the pilot policy from the same screen before confirming.

Evidence ERP benchmark case D1-ambiguous-01 | Demo confirmation

Operational and evaluation limits

In the early prototype, a query returned OS and tags without server IDs. The model labeled 40 named servers anyway, and 22 labels disagreed with the stakeholder rule. P1 computes risk in code and checks explanations against known factors. In a separate D1 P1 batch run, a wave explanation timed out after about 362 seconds of attempts. Template fallback preserved a valid plan, but the full run took 492.6 seconds. Later runs used a shorter timeout and an output cap.

Transfer GB approximates data volume. Bandwidth, replication, and cutover timing still need a model. Current costs are synthetic and AWS estimates cover Linux compute in one region, excluding storage, transfer, support, and license charges. The no-pilot, rule-only local replay split 16 of 38 D1 edges versus 17 in the committed batch result, despite both passing with 10 waves. Graph-engine parity and formal stakeholder ratings remain to be tested.

Evidence Prototype risk evidence | Endpoint incident | Known limitations

Section 6 Remaining Work Before the Final Submission

Priorities reflect the stakeholder's UI feedback, the group's remaining-work plan, and the cost discrepancy observed during replay.

Priority and gap	Work and owner	Completion evidence
Required UI and review	Vraj and Chaitanya. Refine the interface, confirm caller ordering, and coordinate a blind rating of three plans per system.	The stakeholder completes the review workflow and rates usefulness and explainability at least 4/5.
Required evaluation	Dhruvik. Run D3 500, check graph-engine parity, and repeat with at least 10 trials per system and fixed model budgets.	Scorecard shows at least 90% P1 completion, assignment variance, and median and range for setup and planning times.
Required constraints	Dhruvik and Vraj. Require clarification for multiple app matches and extend the benchmark with blackout dates and app windows.	All seven original ambiguous requests clarify and all clear cases still pass.
Required transfer timing	Vraj. Use throughput and daily volume caps to estimate transfer time and check cutover windows.	Worked scenarios show transfer hours and flag waves that exceed their window.
Required cost consistency	Vraj Chaudhari. Refresh FinOps totals after an exclusion.	CSV, dashboard, and summary reconcile on the same active inventory.
Required reproducibility	Tejas. Add CI for tests and the rule-only run. Have a teammate verify setup on a clean checkout.	CI passes and a teammate reproduces the README example.
Required final submission	All four members. Complete the final report and presentation with UI and evaluation screenshots.	Slides and report show the agreed screenshot walkthrough and traceable results.
Optional extensions	Tejas and Dhruvik. Expand regional and license-aware pricing. Vraj. Test the larger model beyond D1.	Pricing assumptions and model tradeoffs are backed by measured comparisons.

Section 7 GitHub Repository and Team Contributions

Repository snapshot main at aa7e93fe0087cb3ae3746c02eb390fa83187398c. System and evaluation evidence entered at e68e8d1.

Project repository | Setup and evaluation instructions

Install the pinned requirements in a Python environment. For the full UI, start Neo4j with docker compose up -d, configure an ASU Voyager key locally, and run streamlit run app.py. To test offline, run python run_p1.py --dataset data/sample.csv --yes --no-llm --no-neo4j. All 42 tests passed on this checkout. The offline smoke test produced a validated 10-wave D1 plan with four documented exceptions.

Team contributions

Dhruvik Chetankumar Asmani (Dhruvik07). Helped plan the project from initial scope through implementation and evaluation. Dhruvik contributed to constraint and evaluation design, collaborated with Tejas, and drafted and revised the Milestone 2 report. Tejas committed and pushed their shared work through his Pegasus1106 account. Dhruvik plans to add and push the completed report to the group repository.

Tejas Rajaram Uttare (Pegasus1106). Collaborated with Dhruvik and maintained the GitHub repository. Tejas built and pushed the initial P1 prototype, including the Neo4j dependency graph, wave planner, FinOps catalog, greedy baseline, and run script. Initial prototype | Setup correction

Chaitanya Muzumdar (0xmzmdr). Helped design the architecture and validation approach, including checks for dependency order, transfer volume, sizing, and risk completeness. Chaitanya also updated the prototype README. README update

Vraj Ashokbhai Chaudhari (vr978). Implemented the LangGraph supervisor, tool-using agents, Streamlit UI, and test suite. Vraj ran the Milestone 2 evaluation and prepared the results and screenshots. Milestone 2 implementation and evidence